CFG for 'matmul_kernel' function

9:

%10 = tail call i32 asm "mov.u32 $0, %ctaid.x %11 = add i32 %3, 31, !dbg !11
%12 = sdiv i32 %11, 32, !dbg !15
%13 = add i32 %4, 63, !dbg !16
%14 = sdiv i32 %13, 64, !dbg !18
%15 = shl nsw i32 %14, 3, !dbg !19
%.frozen = freeze i32 %10
%.frozen230 = freeze i32 %15
%16 = sdiv i32 %.frozen, %.frozen230, !dbg !20
%17 = shl i32 %16, 3, !dbg !21
%18 = sub i32 %12, %17, !dbg !22
%19 = tail call i32 @llvm.smin.i32(i32 %18, i32 8), !dbg !23 %20 = mul i32 %16, %.frozen230
%.decomposed = sub i32 %.frozen, %20
%.frozen231 = freeze i32 %19
%21 = sdiv i32 %.decomposed, %.frozen231, !dbg !24
%22 = mul i32 %21, %.frozen231
%.decomposed232 = sub i32 %.decomposed, %22
%23 = add i32 %.decomposed232, %17, !dbg !25
%24 = shl i32 %23, 5, !dbg !26
%25 = tail call i32 @llvm.nvvm.read.ptx.sreg.tid.x(), !dbg !27 %26 = lshr i32 %25, 5, !dbg !27
%27 = and i32 %25, 4, !dbg !27
%28 = lshr i32 %25, 1, !dbg !27
%29 = and i32 %28, 8, !dbg !27
%30 = shl i32 %21, 6, !dbg !28
%31 = and i32 %25, 1, !dbg !29
%32 = shl nuw nsw i32 %31, 4, !dbg !30
%33 = add i32 %5, 31, !dbg !31
%34 = sdiv i32 %33, 32, !dbg !33
%35 = tail call half asm "cvt.rz.f16.f32 $0, $1
%36 = insertelement <2 x half> poison, half %35, i64 0, !dbg !34 %37 = shufflevector <2 x half> %36, <2 x half> poison, <2 x i32> . . . zeroinitializer, !dbg !34
%38 = bitcast <2 x half> %37 to i32, !dbg !34
%39 = tail call <4 x i8> asm "{ \0A.reg .b32 a<2>
%40 = icmp sgt i32 %33, 31, !dbg !35
br i1 %40, label %.lr.ph, label %.._crit_edge_crit_edge, !dbg !35

T F

. lr.ph:

%41 = and i32 %28, 31, !dbg !29
%42 = or disjoint i32 %41, 32, !dbg !29
%43 = or disjoint i32 %30, %42, !dbg !37
%44 = srem i32 %43, %4, !dbg !38
%45 = mul i32 %44, %7, !dbg !39
%46 = add i32 %45, %32, !dbg !40
%47 = sext i32 %46 to i64, !dbg !41
%48 = getelementptr i8, ptr addrspace(1) %1, i64 %47, !dbg !41 %49 = or disjoint i32 %30, %41, !dbg !37
%50 = srem i32 %49, %4, !dbg !38
%51 = mul i32 %50, %7, !dbg !39
%52 = add i32 %51, %32, !dbg !40
%53 = sext i32 %52 to i64, !dbg !41
%54 = getelementptr i8, ptr addrspace(1) %1, i64 %53, !dbg !41 %55 = or disjoint i32 %24, %41, !dbg !42
%56 = srem i32 %55, %3, !dbg !43
%57 = mul i32 %56, %6, !dbg !44
%58 = add i32 %57, %32, !dbg !45
%59 = sext i32 %58 to i64, !dbg !46
%60 = getelementptr i8, ptr addrspace(1) %0, i64 %59, !dbg !46 %61 = or disjoint i32 %32, 12, !dbg !30
%62 = or disjoint i32 %32, 4, !dbg !30
%63 = and i32 %25, 8, !dbg !27
%64 = bitcast <4 x i8> %39 to i32
%trunc = trunc i32 %25 to i5
%65 = and i5 %trunc, 5
%mask = tail call i5 @llvm.bitreverse.i5(i5 %65)
%66 = zext i5 %mask to i32
%67 = or disjoint i32 %63, %66
%68 = shl nuw nsw i32 %41, 5
%69 = or disjoint i32 %67, %68
%70 = zext nneg i32 %69 to i64
%71 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %70 %72 = xor i32 %62, %27
%73 = or disjoint i32 %72, %63
%74 = or disjoint i32 %73, %68
%75 = zext nneg i32 %74 to i64
%76 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %75 %77 = or disjoint i5 %65, 2
%78 = tail call i5 @llvm.bitreverse.i5(i5 %77)
%79 = zext i5 %78 to i32
%80 = xor i32 %63, %79
%81 = or disjoint i32 %80, %68
%82 = zext nneg i32 %81 to i64
%83 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %82 %and.ra = and i32 %25, 12
%84 = xor i32 %61, %and.ra
%85 = or disjoint i32 %68, %84
%86 = zext nneg i32 %85 to i64
%87 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %86 %88 = shl nuw nsw i32 %26, 4
%89 = and i32 %88, 16
%90 = and i32 %25, 3
%91 = and i32 %25, 7
%92 = or disjoint i32 %91, %89
%93 = or disjoint i32 %92, %29
%94 = shl nuw nsw i32 %91, 1
%95 = and i32 %94, 12
%96 = shl nuw nsw i32 %93, 5
%97 = or disjoint i32 %95, %96
%98 = xor i32 %97, 4
%99 = xor i32 %97, 8
%100 = xor i32 %97, 12
%101 = or disjoint i32 %97, 16
%102 = xor i32 %97, 20
%103 = xor i32 %97, 24
%104 = xor i32 %97, 28
%105 = zext nneg i32 %97 to i64
%106 = getelementptr half, ptr addrspace(3) @global_smem, i64 %105 %107 = zext nneg i32 %98 to i64
%108 = getelementptr half, ptr addrspace(3) @global_smem, i64 %107 %109 = zext nneg i32 %99 to i64
%110 = getelementptr half, ptr addrspace(3) @global_smem, i64 %109 %111 = zext nneg i32 %100 to i64
%112 = getelementptr half, ptr addrspace(3) @global_smem, i64 %111 %113 = zext nneg i32 %101 to i64
%114 = getelementptr half, ptr addrspace(3) @global_smem, i64 %113 %115 = zext nneg i32 %102 to i64
%116 = getelementptr half, ptr addrspace(3) @global_smem, i64 %115 %117 = zext nneg i32 %103 to i64
%118 = getelementptr half, ptr addrspace(3) @global_smem, i64 %117
%119 = zext nneg i32 %104 to i64
%120 = getelementptr half, ptr addrspace(3) @global_smem, i64 %119
%121 = shl nuw nsw i32 %42, 5
%122 = or disjoint i32 %121, %67
%123 = zext nneg i32 %122 to i64
%124 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %123 %125 = or disjoint i32 %121, %73
%126 = zext nneg i32 %125 to i64
%127 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %126 %128 = or disjoint i32 %80, %121
%129 = zext nneg i32 %128 to i64
%130 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %129 %131 = or disjoint i32 %121, %84
%132 = zext nneg i32 %131 to i64
%133 = getelementptr inbounds half, ptr addrspace(3) @global_smem, i64 %132 %134 = and i32 %28, 4
%135 = or disjoint i32 %90, %134
%136 = or disjoint i32 %135, %29
%137 = shl nuw nsw i32 %135, 1
%138 = and i32 %137, 12
%139 = shl nuw nsw i32 %136, 5
%140 = or disjoint i32 %138, %139
%141 = xor i32 %140, 4
%142 = xor i32 %140, 8
%143 = xor i32 %140, 12
%144 = or disjoint i32 %140, 16
%145 = xor i32 %140, 20
%146 = xor i32 %140, 24
%147 = xor i32 %140, 28
%148 = zext nneg i32 %140 to i64
%149 = getelementptr half, ptr addrspace(3) @global_smem, i64 %148 %150 = zext nneg i32 %141 to i64
%151 = getelementptr half, ptr addrspace(3) @global_smem, i64 %150 %152 = zext nneg i32 %142 to i64
%153 = getelementptr half, ptr addrspace(3) @global_smem, i64 %152 %154 = zext nneg i32 %143 to i64
%155 = getelementptr half, ptr addrspace(3) @global_smem, i64 %154 %156 = zext nneg i32 %144 to i64
%157 = getelementptr half, ptr addrspace(3) @global_smem, i64 %156 %158 = zext nneg i32 %145 to i64
%159 = getelementptr half, ptr addrspace(3) @global_smem, i64 %158 %160 = zext nneg i32 %146 to i64
%161 = getelementptr half, ptr addrspace(3) @global_smem, i64 %160 %162 = zext nneg i32 %147 to i64
%163 = getelementptr half, ptr addrspace(3) @global_smem, i64 %162 %164 = getelementptr i8, ptr addrspace(3) %149, i64 1024
%165 = getelementptr i8, ptr addrspace(3) %149, i64 2048
%166 = getelementptr i8, ptr addrspace(3) %149, i64 3072
%167 = getelementptr i8, ptr addrspace(3) %151, i64 1024
%168 = getelementptr i8, ptr addrspace(3) %151, i64 2048
%169 = getelementptr i8, ptr addrspace(3) %151, i64 3072 %170 = getelementptr i8, ptr addrspace(3) %153, i64 1024 %171 = getelementptr i8, ptr addrspace(3) %153, i64 2048 %172 = getelementptr i8, ptr addrspace(3) %153, i64 3072 %173 = getelementptr i8, ptr addrspace(3) %155, i64 1024 %174 = getelementptr i8, ptr addrspace(3) %155, i64 2048 %175 = getelementptr i8, ptr addrspace(3) %155, i64 3072 %176 = getelementptr i8, ptr addrspace(3) %157, i64 1024 %177 = getelementptr i8, ptr addrspace(3) %157, i64 2048 %178 = getelementptr i8, ptr addrspace(3) %157, i64 3072 %179 = getelementptr i8, ptr addrspace(3) %159, i64 1024 %180 = getelementptr i8, ptr addrspace(3) %159, i64 2048 %181 = getelementptr i8, ptr addrspace(3) %159, i64 3072 %182 = getelementptr i8, ptr addrspace(3) %161, i64 1024
%183 = getelementptr i8, ptr addrspace(3) %161, i64 2048
%184 = getelementptr i8, ptr addrspace(3) %161, i64 3072
%185 = getelementptr i8, ptr addrspace(3) %163, i64 1024
%186 = getelementptr i8, ptr addrspace(3) %163, i64 2048
%187 = getelementptr i8, ptr addrspace(3) %163, i64 3072
%188 = getelementptr inbounds i8, ptr addrspace(3) %106, i64 4 %189 = getelementptr inbounds i8, ptr addrspace(3) %108, i64 4 %190 = getelementptr inbounds i8, ptr addrspace(3) %110, i64 4 %191 = getelementptr inbounds i8, ptr addrspace(3) %112, i64 4 %192 = getelementptr inbounds i8, ptr addrspace(3) %114, i64 4 %193 = getelementptr inbounds i8, ptr addrspace(3) %116, i64 4 %194 = getelementptr inbounds i8, ptr addrspace(3) %118, i64 4 %195 = getelementptr inbounds i8, ptr addrspace(3) %120, i64 4 %196 = getelementptr inbounds i8, ptr addrspace(3) %149, i64 4 %197 = getelementptr i8, ptr addrspace(3) %149, i64 1028
%198 = getelementptr i8, ptr addrspace(3) %149, i64 2052
%199 = getelementptr i8, ptr addrspace(3) %149, i64 3076
%200 = getelementptr inbounds i8, ptr addrspace(3) %151, i64 4 %201 = getelementptr i8, ptr addrspace(3) %151, i64 1028
%202 = getelementptr i8, ptr addrspace(3) %151, i64 2052
%203 = getelementptr i8, ptr addrspace(3) %151, i64 3076
%204 = getelementptr inbounds i8, ptr addrspace(3) %153, i64 4 %205 = getelementptr i8, ptr addrspace(3) %153, i64 1028
%206 = getelementptr i8, ptr addrspace(3) %153, i64 2052
%207 = getelementptr i8, ptr addrspace(3) %153, i64 3076
%208 = getelementptr inbounds i8, ptr addrspace(3) %155, i64 4 %209 = getelementptr i8, ptr addrspace(3) %155, i64 1028
%210 = getelementptr i8, ptr addrspace(3) %155, i64 2052
%211 = getelementptr i8, ptr addrspace(3) %155, i64 3076
%212 = getelementptr inbounds i8, ptr addrspace(3) %157, i64 4 %213 = getelementptr i8, ptr addrspace(3) %157, i64 1028
%214 = getelementptr i8, ptr addrspace(3) %157, i64 2052
%215 = getelementptr i8, ptr addrspace(3) %157, i64 3076
%216 = getelementptr inbounds i8, ptr addrspace(3) %159, i64 4 %217 = getelementptr i8, ptr addrspace(3) %159, i64 1028
%218 = getelementptr i8, ptr addrspace(3) %159, i64 2052
%219 = getelementptr i8, ptr addrspace(3) %159, i64 3076
%220 = getelementptr inbounds i8, ptr addrspace(3) %161, i64 4 %221 = getelementptr i8, ptr addrspace(3) %161, i64 1028
%222 = getelementptr i8, ptr addrspace(3) %161, i64 2052
%223 = getelementptr i8, ptr addrspace(3) %161, i64 3076
%224 = getelementptr inbounds i8, ptr addrspace(3) %163, i64 4 %225 = getelementptr i8, ptr addrspace(3) %163, i64 1028
%226 = getelementptr i8, ptr addrspace(3) %163, i64 2052
%227 = getelementptr i8, ptr addrspace(3) %163, i64 3076
br label %228, !dbg !35

. . _crit_edge_crit_edge:

%.pre = and i32 %28, 4, !dbg !36
br label %._crit_edge, !dbg !35

228:

%.pn65134 = phi ptr addrspace(1) [%48, %.lr.ph], [%698, %228] %.pn97133 = phi ptr addrspace(1) [%54, %.lr.ph], [%697, %228] %.pn33132 = phi ptr addrspace(1) [%60, %.lr.ph], [%696, %228] %229 = phi float [0.000000e+00, %.lr.ph], [%661, %228]
%230 = phi float [0.000000e+00, %.lr.ph], [%663, %228]
%231 = phi float [0.000000e+00, %.lr.ph], [%662, %228]
%232 = phi float [0.000000e+00, %.lr.ph], [%664, %228]
%233 = phi float [0.000000e+00, %.lr.ph], [%665, %228]
%234 = phi float [0.000000e+00, %.lr.ph], [%667, %228] %235 = phi float [0.000000e+00, %.lr.ph], [%666, %228] %236 = phi float [0.000000e+00, %.lr.ph], [%668, %228] %237 = phi float [0.000000e+00, %.lr.ph], [%670, %228] %238 = phi float [0.000000e+00, %.lr.ph], [%672, %228] %239 = phi float [0.000000e+00, %.lr.ph], [%671, %228] %240 = phi float [0.000000e+00, %.lr.ph], [%673, %228] %241 = phi float [0.000000e+00, %.lr.ph], [%674, %228] %242 = phi float [0.000000e+00, %.lr.ph], [%676, %228] %243 = phi float [0.000000e+00, %.lr.ph], [%675, %228] %244 = phi float [0.000000e+00, %.lr.ph], [%677, %228] %245 = phi float [0.000000e+00, %.lr.ph], [%679, %228] %246 = phi float [0.000000e+00, %.lr.ph], [%681, %228] %247 = phi float [0.000000e+00, %.lr.ph], [%680, %228] %248 = phi float [0.000000e+00, %.lr.ph], [%682, %228] %249 = phi float [0.000000e+00, %.lr.ph], [%683, %228] %250 = phi float [0.000000e+00, %.lr.ph], [%685, %228] %251 = phi float [0.000000e+00, %.lr.ph], [%684, %228] %252 = phi float [0.000000e+00, %.lr.ph], [%686, %228] %253 = phi float [0.000000e+00, %.lr.ph], [%688, %228] %254 = phi float [0.000000e+00, %.lr.ph], [%690, %228] %255 = phi float [0.000000e+00, %.lr.ph], [%689, %228] %256 = phi float [0.000000e+00, %.lr.ph], [%691, %228] %257 = phi float [0.000000e+00, %.lr.ph], [%692, %228] %258 = phi float [0.000000e+00, %.lr.ph], [%694, %228] %259 = phi float [0.000000e+00, %.lr.ph], [%693, %228]
%260 = phi float [0.000000e+00, %.lr.ph], [%695, %228]
%261 = phi i32 [0, %.lr.ph], [%699, %228]
%262 = shl i32 %261, 5, !dbg !47
%263 = sub i32 %5, %262, !dbg !48
%264 = icmp slt i32 %32, %263, !dbg !49
%265 = tail call { i32, i32, i32, i32 } asm sideeffect "mov.u32 $0, 0x0 %266 = extractvalue { i32, i32, i32, i32 } %265, 0, !dbg !34
%267 = extractvalue { i32, i32, i32, i32 } %265, 1, !dbg !34
%268 = extractvalue { i32, i32, i32, i32 } %265, 2, !dbg !34
%269 = extractvalue { i32, i32, i32, i32 } %265, 3, !dbg !34
%270 = tail call { i32, i32, i32, i32 } asm sideeffect "mov.u32 $0, 0x0 %271 = extractvalue { i32, i32, i32, i32 } %270, 0, !dbg !50
%272 = extractvalue { i32, i32, i32, i32 } %270, 1, !dbg !50
%273 = extractvalue { i32, i32, i32, i32 } %270, 2, !dbg !50
%274 = extractvalue { i32, i32, i32, i32 } %270, 3, !dbg !50
%275 = tail call { i32, i32, i32, i32 } asm sideeffect "mov.u32 $0, 0x0 %276 = extractvalue { i32, i32, i32, i32 } %275, 0, !dbg !50
%277 = extractvalue { i32, i32, i32, i32 } %275, 1, !dbg !50
%278 = extractvalue { i32, i32, i32, i32 } %275, 2, !dbg !50
%279 = extractvalue { i32, i32, i32, i32 } %275, 3, !dbg !50
%280 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%281 = extractvalue { <2 x half>, <2 x half> } %280, 0, !dbg !51 %282 = extractvalue { <2 x half>, <2 x half> } %280, 1, !dbg !51 %283 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%284 = extractvalue { <2 x half>, <2 x half> } %283, 0, !dbg !51 %285 = extractvalue { <2 x half>, <2 x half> } %283, 1, !dbg !51 %286 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%287 = extractvalue { <2 x half>, <2 x half> } %286, 0, !dbg !51 %288 = extractvalue { <2 x half>, <2 x half> } %286, 1, !dbg !51
%289 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%290 = extractvalue { <2 x half>, <2 x half> } %289, 0, !dbg !51
%291 = extractvalue { <2 x half>, <2 x half> } %289, 1, !dbg !51
tail call void @llvm.nvvm.barrier0(), !dbg !51
%292 = shufflevector <2 x half> %281, <2 x half> %282, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %292, ptr addrspace(3) %71, align 8, !dbg !51
%293 = shufflevector <2 x half> %284, <2 x half> %285, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %293, ptr addrspace(3) %76, align 8, !dbg !51
%294 = shufflevector <2 x half> %287, <2 x half> %288, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %294, ptr addrspace(3) %83, align 8, !dbg !51
%295 = shufflevector <2 x half> %290, <2 x half> %291, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %295, ptr addrspace(3) %87, align 8, !dbg !51
tail call void @llvm.nvvm.barrier0(), !dbg !51
%296 = load i32, ptr addrspace(3) %106, align 8, !dbg !51
%297 = load i32, ptr addrspace(3) %188, align 4, !dbg !51
%298 = load i32, ptr addrspace(3) %108, align 8, !dbg !51 %299 = load i32, ptr addrspace(3) %189, align 4, !dbg !51 %300 = load i32, ptr addrspace(3) %110, align 8, !dbg !51 %301 = load i32, ptr addrspace(3) %190, align 4, !dbg !51 %302 = load i32, ptr addrspace(3) %112, align 8, !dbg !51 %303 = load i32, ptr addrspace(3) %191, align 4, !dbg !51 %304 = load i32, ptr addrspace(3) %114, align 8, !dbg !51 %305 = load i32, ptr addrspace(3) %192, align 4, !dbg !51 %306 = load i32, ptr addrspace(3) %116, align 8, !dbg !51 %307 = load i32, ptr addrspace(3) %193, align 4, !dbg !51 %308 = load i32, ptr addrspace(3) %118, align 8, !dbg !51
%309 = load i32, ptr addrspace(3) %194, align 4, !dbg !51
%310 = load i32, ptr addrspace(3) %120, align 8, !dbg !51
%311 = load i32, ptr addrspace(3) %195, align 4, !dbg !51
%312 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%313 = extractvalue { <2 x half>, <2 x half> } %312, 0, !dbg !51 %314 = extractvalue { <2 x half>, <2 x half> } %312, 1, !dbg !51 %315 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%316 = extractvalue { <2 x half>, <2 x half> } %315, 0, !dbg !51 %317 = extractvalue { <2 x half>, <2 x half> } %315, 1, !dbg !51 %318 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%319 = extractvalue { <2 x half>, <2 x half> } %318, 0, !dbg !51 %320 = extractvalue { <2 x half>, <2 x half> } %318, 1, !dbg !51 %321 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%322 = extractvalue { <2 x half>, <2 x half> } %321, 0, !dbg !51 %323 = extractvalue { <2 x half>, <2 x half> } %321, 1, !dbg !51 %324 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%325 = extractvalue { <2 x half>, <2 x half> } %324, 0, !dbg !51 %326 = extractvalue { <2 x half>, <2 x half> } %324, 1, !dbg !51 %327 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%328 = extractvalue { <2 x half>, <2 x half> } %327, 0, !dbg !51 %329 = extractvalue { <2 x half>, <2 x half> } %327, 1, !dbg !51 %330 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%331 = extractvalue { <2 x half>, <2 x half> } %330, 0, !dbg !51 %332 = extractvalue { <2 x half>, <2 x half> } %330, 1, !dbg !51
%333 = tail call { <2 x half>, <2 x half> } asm "{
. . . \0Aprmt.b32 $0, 0, $2, 0x5140
%334 = extractvalue { <2 x half>, <2 x half> } %333, 0, !dbg !51
%335 = extractvalue { <2 x half>, <2 x half> } %333, 1, !dbg !51
tail call void @llvm.nvvm.barrier0(), !dbg !51
%336 = shufflevector <2 x half> %313, <2 x half> %314, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %336, ptr addrspace(3) %71, align 8, !dbg !51
%337 = shufflevector <2 x half> %316, <2 x half> %317, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %337, ptr addrspace(3) %76, align 8, !dbg !51
%338 = shufflevector <2 x half> %319, <2 x half> %320, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %338, ptr addrspace(3) %83, align 8, !dbg !51
%339 = shufflevector <2 x half> %322, <2 x half> %323, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %339, ptr addrspace(3) %87, align 8, !dbg !51
%340 = shufflevector <2 x half> %325, <2 x half> %326, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %340, ptr addrspace(3) %124, align 8, !dbg !51
%341 = shufflevector <2 x half> %328, <2 x half> %329, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %341, ptr addrspace(3) %127, align 8, !dbg !51
%342 = shufflevector <2 x half> %331, <2 x half> %332, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %342, ptr addrspace(3) %130, align 8, !dbg !51
%343 = shufflevector <2 x half> %334, <2 x half> %335, <4 x i32> <i32 0, i32
. . . 1, i32 2, i32 3>, !dbg !51
store <4 x half> %343, ptr addrspace(3) %133, align 8, !dbg !51
tail call void @llvm.nvvm.barrier0(), !dbg !51
%344 = load i32, ptr addrspace(3) %149, align 8, !dbg !51
%345 = load i32, ptr addrspace(3) %196, align 4, !dbg !51
%346 = load i32, ptr addrspace(3) %164, align 8, !dbg !51
%347 = load i32, ptr addrspace(3) %197, align 4, !dbg !51
%348 = load i32, ptr addrspace(3) %165, align 8, !dbg !51 %349 = load i32, ptr addrspace(3) %198, align 4, !dbg !51 %350 = load i32, ptr addrspace(3) %166, align 8, !dbg !51 %351 = load i32, ptr addrspace(3) %199, align 4, !dbg !51 %352 = load i32, ptr addrspace(3) %151, align 8, !dbg !51 %353 = load i32, ptr addrspace(3) %200, align 4, !dbg !51 %354 = load i32, ptr addrspace(3) %167, align 8, !dbg !51 %355 = load i32, ptr addrspace(3) %201, align 4, !dbg !51 %356 = load i32, ptr addrspace(3) %168, align 8, !dbg !51 %357 = load i32, ptr addrspace(3) %202, align 4, !dbg !51 %358 = load i32, ptr addrspace(3) %169, align 8, !dbg !51 %359 = load i32, ptr addrspace(3) %203, align 4, !dbg !51 %360 = load i32, ptr addrspace(3) %153, align 8, !dbg !51 %361 = load i32, ptr addrspace(3) %204, align 4, !dbg !51 %362 = load i32, ptr addrspace(3) %170, align 8, !dbg !51 %363 = load i32, ptr addrspace(3) %205, align 4, !dbg !51 %364 = load i32, ptr addrspace(3) %171, align 8, !dbg !51 %365 = load i32, ptr addrspace(3) %206, align 4, !dbg !51 %366 = load i32, ptr addrspace(3) %172, align 8, !dbg !51 %367 = load i32, ptr addrspace(3) %207, align 4, !dbg !51 %368 = load i32, ptr addrspace(3) %155, align 8, !dbg !51 %369 = load i32, ptr addrspace(3) %208, align 4, !dbg !51 %370 = load i32, ptr addrspace(3) %173, align 8, !dbg !51 %371 = load i32, ptr addrspace(3) %209, align 4, !dbg !51 %372 = load i32, ptr addrspace(3) %174, align 8, !dbg !51 %373 = load i32, ptr addrspace(3) %210, align 4, !dbg !51 %374 = load i32, ptr addrspace(3) %175, align 8, !dbg !51 %375 = load i32, ptr addrspace(3) %211, align 4, !dbg !51 %376 = load i32, ptr addrspace(3) %157, align 8, !dbg !51 %377 = load i32, ptr addrspace(3) %212, align 4, !dbg !51 %378 = load i32, ptr addrspace(3) %176, align 8, !dbg !51 %379 = load i32, ptr addrspace(3) %213, align 4, !dbg !51 %380 = load i32, ptr addrspace(3) %177, align 8, !dbg !51 %381 = load i32, ptr addrspace(3) %214, align 4, !dbg !51 %382 = load i32, ptr addrspace(3) %178, align 8, !dbg !51 %383 = load i32, ptr addrspace(3) %215, align 4, !dbg !51 %384 = load i32, ptr addrspace(3) %159, align 8, !dbg !51 %385 = load i32, ptr addrspace(3) %216, align 4, !dbg !51 %386 = load i32, ptr addrspace(3) %179, align 8, !dbg !51 %387 = load i32, ptr addrspace(3) %217, align 4, !dbg !51 %388 = load i32, ptr addrspace(3) %180, align 8, !dbg !51 %389 = load i32, ptr addrspace(3) %218, align 4, !dbg !51 %390 = load i32, ptr addrspace(3) %181, align 8, !dbg !51 %391 = load i32, ptr addrspace(3) %219, align 4, !dbg !51 %392 = load i32, ptr addrspace(3) %161, align 8, !dbg !51 %393 = load i32, ptr addrspace(3) %220, align 4, !dbg !51 %394 = load i32, ptr addrspace(3) %182, align 8, !dbg !51 %395 = load i32, ptr addrspace(3) %221, align 4, !dbg !51 %396 = load i32, ptr addrspace(3) %183, align 8, !dbg !51 %397 = load i32, ptr addrspace(3) %222, align 4, !dbg !51 %398 = load i32, ptr addrspace(3) %184, align 8, !dbg !51 %399 = load i32, ptr addrspace(3) %223, align 4, !dbg !51 %400 = load i32, ptr addrspace(3) %163, align 8, !dbg !51 %401 = load i32, ptr addrspace(3) %224, align 4, !dbg !51 %402 = load i32, ptr addrspace(3) %185, align 8, !dbg !51
%403 = load i32, ptr addrspace(3) %225, align 4, !dbg !51
%404 = load i32, ptr addrspace(3) %186, align 8, !dbg !51
%405 = load i32, ptr addrspace(3) %226, align 4, !dbg !51
%406 = load i32, ptr addrspace(3) %187, align 8, !dbg !51
%407 = load i32, ptr addrspace(3) %227, align 4, !dbg !51
%408 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%409 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 0, !dbg !51
%410 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 1, !dbg !51
%411 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 2, !dbg !51
%412 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 3, !dbg !51
%413 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 4, !dbg !51
%414 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 5, !dbg !51
%415 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 6, !dbg !51
%416 = extractvalue { float, float, float, float, float, float, float, float
. . . } %408, 7, !dbg !51
%417 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%418 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 0, !dbg !51
%419 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 1, !dbg !51
%420 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 2, !dbg !51
%421 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 3, !dbg !51
%422 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 4, !dbg !51
%423 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 5, !dbg !51
%424 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 6, !dbg !51
%425 = extractvalue { float, float, float, float, float, float, float, float
. . . } %417, 7, !dbg !51
%426 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%427 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 0, !dbg !51
%428 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 1, !dbg !51
%429 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 2, !dbg !51
%430 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 3, !dbg !51
%431 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 4, !dbg !51
%432 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 5, !dbg !51
%433 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 6, !dbg !51
%434 = extractvalue { float, float, float, float, float, float, float, float
. . . } %426, 7, !dbg !51
%435 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%436 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 0, !dbg !51
%437 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 1, !dbg !51
%438 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 2, !dbg !51
%439 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 3, !dbg !51
%440 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 4, !dbg !51
%441 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 5, !dbg !51
%442 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 6, !dbg !51
%443 = extractvalue { float, float, float, float, float, float, float, float
. . . } %435, 7, !dbg !51
%444 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%445 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 0, !dbg !51
%446 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 1, !dbg !51
%447 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 2, !dbg !51
%448 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 3, !dbg !51
%449 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 4, !dbg !51
%450 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 5, !dbg !51
%451 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 6, !dbg !51
%452 = extractvalue { float, float, float, float, float, float, float, float
. . . } %444, 7, !dbg !51
%453 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%454 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 0, !dbg !51
%455 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 1, !dbg !51
%456 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 2, !dbg !51
%457 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 3, !dbg !51
%458 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 4, !dbg !51
%459 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 5, !dbg !51
%460 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 6, !dbg !51
%461 = extractvalue { float, float, float, float, float, float, float, float
. . . } %453, 7, !dbg !51
%462 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%463 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 0, !dbg !51
%464 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 1, !dbg !51
%465 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 2, !dbg !51
%466 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 3, !dbg !51
%467 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 4, !dbg !51
%468 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 5, !dbg !51
%469 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 6, !dbg !51
%470 = extractvalue { float, float, float, float, float, float, float, float
. . . } %462, 7, !dbg !51
%471 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%472 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 0, !dbg !51
%473 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 1, !dbg !51
%474 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 2, !dbg !51
%475 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 3, !dbg !51
%476 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 4, !dbg !51
%477 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 5, !dbg !51
%478 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 6, !dbg !51
%479 = extractvalue { float, float, float, float, float, float, float, float
. . . } %471, 7, !dbg !51
%480 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%481 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 0, !dbg !51
%482 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 1, !dbg !51
%483 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 2, !dbg !51
%484 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 3, !dbg !51
%485 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 4, !dbg !51
%486 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 5, !dbg !51
%487 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 6, !dbg !51
%488 = extractvalue { float, float, float, float, float, float, float, float
. . . } %480, 7, !dbg !51
%489 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%490 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 0, !dbg !51
%491 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 1, !dbg !51
%492 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 2, !dbg !51
%493 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 3, !dbg !51
%494 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 4, !dbg !51
%495 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 5, !dbg !51
%496 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 6, !dbg !51
%497 = extractvalue { float, float, float, float, float, float, float, float
. . . } %489, 7, !dbg !51
%498 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%499 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 0, !dbg !51
%500 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 1, !dbg !51
%501 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 2, !dbg !51
%502 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 3, !dbg !51
%503 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 4, !dbg !51
%504 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 5, !dbg !51
%505 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 6, !dbg !51
%506 = extractvalue { float, float, float, float, float, float, float, float
. . . } %498, 7, !dbg !51
%507 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%508 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 0, !dbg !51
%509 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 1, !dbg !51
%510 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 2, !dbg !51
%511 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 3, !dbg !51
%512 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 4, !dbg !51
%513 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 5, !dbg !51
%514 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 6, !dbg !51
%515 = extractvalue { float, float, float, float, float, float, float, float
. . . } %507, 7, !dbg !51
%516 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%517 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 0, !dbg !51
%518 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 1, !dbg !51
%519 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 2, !dbg !51
%520 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 3, !dbg !51
%521 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 4, !dbg !51
%522 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 5, !dbg !51
%523 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 6, !dbg !51
%524 = extractvalue { float, float, float, float, float, float, float, float
. . . } %516, 7, !dbg !51
%525 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%526 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 0, !dbg !51
%527 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 1, !dbg !51
%528 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 2, !dbg !51
%529 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 3, !dbg !51
%530 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 4, !dbg !51
%531 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 5, !dbg !51
%532 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 6, !dbg !51
%533 = extractvalue { float, float, float, float, float, float, float, float
. . . } %525, 7, !dbg !51
%534 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%535 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 0, !dbg !51
%536 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 1, !dbg !51
%537 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 2, !dbg !51
%538 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 3, !dbg !51
%539 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 4, !dbg !51
%540 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 5, !dbg !51
%541 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 6, !dbg !51
%542 = extractvalue { float, float, float, float, float, float, float, float
. . . } %534, 7, !dbg !51
%543 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%544 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 0, !dbg !51
%545 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 1, !dbg !51
%546 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 2, !dbg !51
%547 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 3, !dbg !51
%548 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 4, !dbg !51
%549 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 5, !dbg !51
%550 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 6, !dbg !51
%551 = extractvalue { float, float, float, float, float, float, float, float
. . . } %543, 7, !dbg !51
%552 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%553 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 0, !dbg !51
%554 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 1, !dbg !51
%555 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 2, !dbg !51
%556 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 3, !dbg !51
%557 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 4, !dbg !51
%558 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 5, !dbg !51
%559 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 6, !dbg !51
%560 = extractvalue { float, float, float, float, float, float, float, float
. . . } %552, 7, !dbg !51
%561 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%562 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 0, !dbg !51
%563 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 1, !dbg !51
%564 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 2, !dbg !51
%565 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 3, !dbg !51
%566 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 4, !dbg !51
%567 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 5, !dbg !51
%568 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 6, !dbg !51
%569 = extractvalue { float, float, float, float, float, float, float, float
. . . } %561, 7, !dbg !51
%570 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%571 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 0, !dbg !51
%572 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 1, !dbg !51
%573 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 2, !dbg !51
%574 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 3, !dbg !51
%575 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 4, !dbg !51
%576 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 5, !dbg !51
%577 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 6, !dbg !51
%578 = extractvalue { float, float, float, float, float, float, float, float
. . . } %570, 7, !dbg !51
%579 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%580 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 0, !dbg !51
%581 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 1, !dbg !51
%582 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 2, !dbg !51
%583 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 3, !dbg !51
%584 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 4, !dbg !51
%585 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 5, !dbg !51
%586 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 6, !dbg !51
%587 = extractvalue { float, float, float, float, float, float, float, float
. . . } %579, 7, !dbg !51
%588 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%589 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 0, !dbg !51
%590 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 1, !dbg !51
%591 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 2, !dbg !51
%592 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 3, !dbg !51
%593 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 4, !dbg !51
%594 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 5, !dbg !51
%595 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 6, !dbg !51
%596 = extractvalue { float, float, float, float, float, float, float, float
. . . } %588, 7, !dbg !51
%597 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%598 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 0, !dbg !51
%599 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 1, !dbg !51
%600 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 2, !dbg !51
%601 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 3, !dbg !51
%602 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 4, !dbg !51
%603 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 5, !dbg !51
%604 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 6, !dbg !51
%605 = extractvalue { float, float, float, float, float, float, float, float
. . . } %597, 7, !dbg !51
%606 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%607 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 0, !dbg !51
%608 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 1, !dbg !51
%609 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 2, !dbg !51
%610 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 3, !dbg !51
%611 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 4, !dbg !51
%612 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 5, !dbg !51
%613 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 6, !dbg !51
%614 = extractvalue { float, float, float, float, float, float, float, float
. . . } %606, 7, !dbg !51
%615 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%616 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 0, !dbg !51
%617 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 1, !dbg !51
%618 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 2, !dbg !51
%619 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 3, !dbg !51
%620 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 4, !dbg !51
%621 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 5, !dbg !51
%622 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 6, !dbg !51
%623 = extractvalue { float, float, float, float, float, float, float, float
. . . } %615, 7, !dbg !51
%624 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%625 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 0, !dbg !51
%626 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 1, !dbg !51
%627 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 2, !dbg !51
%628 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 3, !dbg !51
%629 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 4, !dbg !51
%630 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 5, !dbg !51
%631 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 6, !dbg !51
%632 = extractvalue { float, float, float, float, float, float, float, float
. . . } %624, 7, !dbg !51
%633 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%634 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 0, !dbg !51
%635 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 1, !dbg !51
%636 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 2, !dbg !51
%637 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 3, !dbg !51
%638 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 4, !dbg !51
%639 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 5, !dbg !51
%640 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 6, !dbg !51
%641 = extractvalue { float, float, float, float, float, float, float, float
. . . } %633, 7, !dbg !51
%642 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%643 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 0, !dbg !51
%644 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 1, !dbg !51
%645 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 2, !dbg !51
%646 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 3, !dbg !51
%647 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 4, !dbg !51
%648 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 5, !dbg !51
%649 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 6, !dbg !51
%650 = extractvalue { float, float, float, float, float, float, float, float
. . . } %642, 7, !dbg !51
%651 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%652 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 0, !dbg !51
%653 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 1, !dbg !51
%654 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 2, !dbg !51
%655 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 3, !dbg !51
%656 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 4, !dbg !51
%657 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 5, !dbg !51
%658 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 6, !dbg !51
%659 = extractvalue { float, float, float, float, float, float, float, float
. . . } %651, 7, !dbg !51
%660 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%661 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 0, !dbg !51
%662 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 1, !dbg !51
%663 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 2, !dbg !51
%664 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 3, !dbg !51
%665 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 4, !dbg !51
%666 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 5, !dbg !51
%667 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 6, !dbg !51
%668 = extractvalue { float, float, float, float, float, float, float, float
. . . } %660, 7, !dbg !51
%669 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%670 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 0, !dbg !51
%671 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 1, !dbg !51
%672 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 2, !dbg !51
%673 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 3, !dbg !51
%674 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 4, !dbg !51
%675 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 5, !dbg !51
%676 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 6, !dbg !51
%677 = extractvalue { float, float, float, float, float, float, float, float
. . . } %669, 7, !dbg !51
%678 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%679 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 0, !dbg !51
%680 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 1, !dbg !51
%681 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 2, !dbg !51
%682 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 3, !dbg !51
%683 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 4, !dbg !51
%684 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 5, !dbg !51
%685 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 6, !dbg !51
%686 = extractvalue { float, float, float, float, float, float, float, float
. . . } %678, 7, !dbg !51
%687 = tail call { float, float, float, float, float, float, float, float }
. . . asm sideeffect "mma.sync.aligned.m8n8k4.row.col.f32.f16.f16.f32 { $0, $1, $2,
. . . $3, $4, $5, $6, $7 }, { $8, $9 }, { $10, $11 }, { $12, $13, $14, $15, $16,
. . . $17, $18, $19 }
%688 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 0, !dbg !51
%689 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 1, !dbg !51
%690 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 2, !dbg !51
%691 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 3, !dbg !51
%692 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 4, !dbg !51
%693 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 5, !dbg !51
%694 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 6, !dbg !51
%695 = extractvalue { float, float, float, float, float, float, float, float
. . . } %687, 7, !dbg !51
%696 = getelementptr i8, ptr addrspace(1) %.pn33132, i64 32, !dbg !52
%697 = getelementptr i8, ptr addrspace(1) %.pn97133, i64 32, !dbg !53
%698 = getelementptr i8, ptr addrspace(1) %.pn65134, i64 32, !dbg !53
%699 = add nuw nsw i32 %261, 1, !dbg !35
%700 = icmp slt i32 %699, %34, !dbg !35
br i1 %700, label %228, label %._crit_edge.loopexit, !dbg !35

T F

. _crit_edge:

%.pre-phi = phi i32 [%.pre, %.._crit_edge_crit_edge], [%134,
. . . %._crit_edge.loopexit], !dbg !36
%734 = phi <32 x half> [zeroinitializer, %.._crit_edge_crit_edge], [%733,
. . . %._crit_edge.loopexit]
%735 = and i32 %25, 2, !dbg !27
%736 = shl i32 %25, 3, !dbg !29
%737 = and i32 %736, 56, !dbg !29
%738 = or disjoint i32 %30, %737, !dbg !37
%739 = lshr i32 %25, 3, !dbg !27
%740 = and i32 %739, 7, !dbg !27
%741 = or disjoint i32 %740, %24, !dbg !42 %742 = or disjoint i32 %741, 24, !dbg !42 %743 = or disjoint i32 %741, 16, !dbg !42 %744 = or disjoint i32 %741, 8, !dbg !42
%745 = mul i32 %741, %8, !dbg !55
%746 = mul i32 %744, %8, !dbg !55
%747 = mul i32 %743, %8, !dbg !55
%748 = mul i32 %742, %8, !dbg !55
%749 = sext i32 %745 to i64, !dbg !56
%750 = getelementptr half, ptr addrspace(1) %2, i64 %749, !dbg !56 %751 = sext i32 %746 to i64, !dbg !56
%752 = getelementptr half, ptr addrspace(1) %2, i64 %751, !dbg !56 %753 = sext i32 %747 to i64, !dbg !56
%754 = getelementptr half, ptr addrspace(1) %2, i64 %753, !dbg !56 %755 = sext i32 %748 to i64, !dbg !56
%756 = getelementptr half, ptr addrspace(1) %2, i64 %755, !dbg !56 %757 = sext i32 %738 to i64, !dbg !57
%758 = getelementptr half, ptr addrspace(1) %750, i64 %757, !dbg !57 %759 = getelementptr half, ptr addrspace(1) %752, i64 %757, !dbg !57 %760 = getelementptr half, ptr addrspace(1) %754, i64 %757, !dbg !57 %761 = getelementptr half, ptr addrspace(1) %756, i64 %757, !dbg !57 %762 = icmp slt i32 %741, %3, !dbg !58
%763 = icmp slt i32 %744, %3, !dbg !58
%764 = icmp slt i32 %743, %3, !dbg !58
%765 = icmp slt i32 %742, %3, !dbg !58
%766 = icmp slt i32 %738, %4, !dbg !59
%767 = and i1 %762, %766, !dbg !60
%768 = and i1 %763, %766, !dbg !60
%769 = and i1 %764, %766, !dbg !60
%770 = and i1 %765, %766, !dbg !60
tail call void @llvm.nvvm.barrier0(), !dbg !36 %771 = and i32 %26, 1, !dbg !36
%772 = shl nuw nsw i32 %771, 4, !dbg !36
%773 = or disjoint i32 %772, %29, !dbg !36
%774 = or disjoint i32 %773, %27, !dbg !36
%775 = or disjoint i32 %774, %31, !dbg !36
%776 = or disjoint i32 %.pre-phi, %735, !dbg !36 %777 = or disjoint i32 %776, 8, !dbg !36
%778 = or disjoint i32 %776, 16, !dbg !36
%779 = or disjoint i32 %776, 24, !dbg !36
%780 = or disjoint i32 %776, 32, !dbg !36
%781 = or disjoint i32 %776, 40, !dbg !36
%782 = or disjoint i32 %776, 48, !dbg !36
%783 = or disjoint i32 %776, 56, !dbg !36
%784 = mul nuw nsw i32 %775, 72, !dbg !36
%785 = or disjoint i32 %784, %776, !dbg !36
%786 = zext nneg i32 %785 to i64, !dbg !36
%787 = getelementptr half, ptr addrspace(3) @global_smem, i64 %786, !dbg !36 %788 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32 0,
. . . i32 2>, !dbg !36
store <2 x half> %788, ptr addrspace(3) %787, align 4, !dbg !36
%789 = add nuw nsw i32 %784, %777, !dbg !36
%790 = zext nneg i32 %789 to i64, !dbg !36
%791 = getelementptr half, ptr addrspace(3) @global_smem, i64 %790, !dbg !36 %792 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32 4,
. . . i32 6>, !dbg !36
store <2 x half> %792, ptr addrspace(3) %791, align 4, !dbg !36
%793 = add nuw nsw i32 %784, %778, !dbg !36
%794 = zext nneg i32 %793 to i64, !dbg !36
%795 = getelementptr half, ptr addrspace(3) @global_smem, i64 %794, !dbg !36 %796 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32 8,
. . . i32 10>, !dbg !36
store <2 x half> %796, ptr addrspace(3) %795, align 4, !dbg !36
%797 = add nuw nsw i32 %784, %779, !dbg !36
%798 = zext nneg i32 %797 to i64, !dbg !36
%799 = getelementptr half, ptr addrspace(3) @global_smem, i64 %798, !dbg !36 %800 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 12, i32 14>, !dbg !36
store <2 x half> %800, ptr addrspace(3) %799, align 4, !dbg !36
%801 = add nuw nsw i32 %784, %780, !dbg !36
%802 = zext nneg i32 %801 to i64, !dbg !36
%803 = getelementptr half, ptr addrspace(3) @global_smem, i64 %802, !dbg !36 %804 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 16, i32 18>, !dbg !36
store <2 x half> %804, ptr addrspace(3) %803, align 4, !dbg !36
%805 = add nuw nsw i32 %784, %781, !dbg !36
%806 = zext nneg i32 %805 to i64, !dbg !36
%807 = getelementptr half, ptr addrspace(3) @global_smem, i64 %806, !dbg !36 %808 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 20, i32 22>, !dbg !36
store <2 x half> %808, ptr addrspace(3) %807, align 4, !dbg !36
%809 = add nuw nsw i32 %784, %782, !dbg !36
%810 = zext nneg i32 %809 to i64, !dbg !36
%811 = getelementptr half, ptr addrspace(3) @global_smem, i64 %810, !dbg !36 %812 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 24, i32 26>, !dbg !36
store <2 x half> %812, ptr addrspace(3) %811, align 4, !dbg !36
%813 = add nuw nsw i32 %784, %783, !dbg !36
%814 = zext nneg i32 %813 to i64, !dbg !36
%815 = getelementptr half, ptr addrspace(3) @global_smem, i64 %814, !dbg !36 %816 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 28, i32 30>, !dbg !36
store <2 x half> %816, ptr addrspace(3) %815, align 4, !dbg !36
%817 = add nuw nsw i32 %784, 144, !dbg !36
%818 = or disjoint i32 %817, %776, !dbg !36
%819 = zext nneg i32 %818 to i64
%820 = getelementptr half, ptr addrspace(3) @global_smem, i64 %819, !dbg !36 %821 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32 1,
. . . i32 3>, !dbg !36
store <2 x half> %821, ptr addrspace(3) %820, align 4, !dbg !36
%822 = add nuw nsw i32 %817, %777, !dbg !36
%823 = zext nneg i32 %822 to i64
%824 = getelementptr half, ptr addrspace(3) @global_smem, i64 %823, !dbg !36 %825 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32 5,
. . . i32 7>, !dbg !36
store <2 x half> %825, ptr addrspace(3) %824, align 4, !dbg !36
%826 = add nuw nsw i32 %817, %778, !dbg !36
%827 = zext nneg i32 %826 to i64
%828 = getelementptr half, ptr addrspace(3) @global_smem, i64 %827, !dbg !36 %829 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32 9,
. . . i32 11>, !dbg !36
store <2 x half> %829, ptr addrspace(3) %828, align 4, !dbg !36
%830 = add nuw nsw i32 %817, %779, !dbg !36
%831 = zext nneg i32 %830 to i64
%832 = getelementptr half, ptr addrspace(3) @global_smem, i64 %831, !dbg !36 %833 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 13, i32 15>, !dbg !36
store <2 x half> %833, ptr addrspace(3) %832, align 4, !dbg !36
%834 = add nuw nsw i32 %817, %780, !dbg !36
%835 = zext nneg i32 %834 to i64
%836 = getelementptr half, ptr addrspace(3) @global_smem, i64 %835, !dbg !36 %837 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 17, i32 19>, !dbg !36
store <2 x half> %837, ptr addrspace(3) %836, align 4, !dbg !36
%838 = add nuw nsw i32 %817, %781, !dbg !36
%839 = zext nneg i32 %838 to i64
%840 = getelementptr half, ptr addrspace(3) @global_smem, i64 %839, !dbg !36 %841 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 21, i32 23>, !dbg !36
store <2 x half> %841, ptr addrspace(3) %840, align 4, !dbg !36
%842 = add nuw nsw i32 %817, %782, !dbg !36
%843 = zext nneg i32 %842 to i64
%844 = getelementptr half, ptr addrspace(3) @global_smem, i64 %843, !dbg !36 %845 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 25, i32 27>, !dbg !36
store <2 x half> %845, ptr addrspace(3) %844, align 4, !dbg !36
%846 = add nuw nsw i32 %817, %783, !dbg !36
%847 = zext nneg i32 %846 to i64
%848 = getelementptr half, ptr addrspace(3) @global_smem, i64 %847, !dbg !36 %849 = shufflevector <32 x half> %734, <32 x half> poison, <2 x i32> <i32
. . . 29, i32 31>, !dbg !36
store <2 x half> %849, ptr addrspace(3) %848, align 4, !dbg !36
tail call void @llvm.nvvm.barrier0(), !dbg !36
%850 = and i32 %739, 3, !dbg !36
%851 = shl nuw nsw i32 %771, 2, !dbg !36
%852 = or disjoint i32 %851, %850, !dbg !36
%853 = mul nuw nsw i32 %852, 72, !dbg !36
%854 = add nuw nsw i32 %853, %737, !dbg !36
%855 = zext nneg i32 %854 to i64, !dbg !36
%856 = getelementptr half, ptr addrspace(3) @global_smem, i64 %855, !dbg !36 %857 = getelementptr i8, ptr addrspace(3) %856, i64 1152, !dbg !36
%858 = load <4 x i32>, ptr addrspace(3) %857, align 16, !dbg !36
%859 = getelementptr i8, ptr addrspace(3) %856, i64 2304, !dbg !36
%860 = load <4 x i32>, ptr addrspace(3) %859, align 16, !dbg !36
%861 = getelementptr i8, ptr addrspace(3) %856, i64 3456, !dbg !36
%862 = load <4 x i32>, ptr addrspace(3) %861, align 16, !dbg !36 %.extract = load i32, ptr addrspace(3) %856, align 16, !dbg !36
%863 = getelementptr inbounds i8, ptr addrspace(3) %856, i64 4, !dbg !36 %.extract99 = load i32, ptr addrspace(3) %863, align 4, !dbg !36
%864 = getelementptr inbounds i8, ptr addrspace(3) %856, i64 8, !dbg !36 %.extract101 = load i32, ptr addrspace(3) %864, align 8, !dbg !36
%865 = getelementptr inbounds i8, ptr addrspace(3) %856, i64 12, !dbg !36 %.extract103 = load i32, ptr addrspace(3) %865, align 4, !dbg !36
tail call void asm sideeffect "@$5 st.global.v4.b32 [$4 + 0], { $0, $1,
. . . $2, $3 }
%.extract105 = extractelement <4 x i32> %858, i64 0, !dbg !36 %.extract107 = extractelement <4 x i32> %858, i64 1, !dbg !36 %.extract109 = extractelement <4 x i32> %858, i64 2, !dbg !36 %.extract111 = extractelement <4 x i32> %858, i64 3, !dbg !36
tail call void asm sideeffect "@$5 st.global.v4.b32 [$4 + 0], { $0, $1,
. . . $2, $3 }
%.extract113 = extractelement <4 x i32> %860, i64 0, !dbg !36 %.extract115 = extractelement <4 x i32> %860, i64 1, !dbg !36 %.extract117 = extractelement <4 x i32> %860, i64 2, !dbg !36 %.extract119 = extractelement <4 x i32> %860, i64 3, !dbg !36
tail call void asm sideeffect "@$5 st.global.v4.b32 [$4 + 0], { $0, $1,
. . . $2, $3 }
%.extract121 = extractelement <4 x i32> %862, i64 0, !dbg !36 %.extract123 = extractelement <4 x i32> %862, i64 1, !dbg !36 %.extract125 = extractelement <4 x i32> %862, i64 2, !dbg !36 %.extract127 = extractelement <4 x i32> %862, i64 3, !dbg !36
tail call void asm sideeffect "@$5 st.global.v4.b32 [$4 + 0], { $0, $1,
. . . $2, $3 }
ret void, !dbg !61

. _crit_edge.loopexit:

%701 = insertelement <32 x float> poison, float %661, i64 0, !dbg !54 %702 = insertelement <32 x float> %701, float %663, i64 1, !dbg !54 %703 = insertelement <32 x float> %702, float %662, i64 2, !dbg !54 %704 = insertelement <32 x float> %703, float %664, i64 3, !dbg !54 %705 = insertelement <32 x float> %704, float %665, i64 4, !dbg !54 %706 = insertelement <32 x float> %705, float %667, i64 5, !dbg !54 %707 = insertelement <32 x float> %706, float %666, i64 6, !dbg !54 %708 = insertelement <32 x float> %707, float %668, i64 7, !dbg !54 %709 = insertelement <32 x float> %708, float %670, i64 8, !dbg !54 %710 = insertelement <32 x float> %709, float %672, i64 9, !dbg !54 %711 = insertelement <32 x float> %710, float %671, i64 10, !dbg !54 %712 = insertelement <32 x float> %711, float %673, i64 11, !dbg !54 %713 = insertelement <32 x float> %712, float %674, i64 12, !dbg !54 %714 = insertelement <32 x float> %713, float %676, i64 13, !dbg !54 %715 = insertelement <32 x float> %714, float %675, i64 14, !dbg !54 %716 = insertelement <32 x float> %715, float %677, i64 15, !dbg !54 %717 = insertelement <32 x float> %716, float %679, i64 16, !dbg !54 %718 = insertelement <32 x float> %717, float %681, i64 17, !dbg !54 %719 = insertelement <32 x float> %718, float %680, i64 18, !dbg !54 %720 = insertelement <32 x float> %719, float %682, i64 19, !dbg !54 %721 = insertelement <32 x float> %720, float %683, i64 20, !dbg !54 %722 = insertelement <32 x float> %721, float %685, i64 21, !dbg !54 %723 = insertelement <32 x float> %722, float %684, i64 22, !dbg !54 %724 = insertelement <32 x float> %723, float %686, i64 23, !dbg !54 %725 = insertelement <32 x float> %724, float %688, i64 24, !dbg !54 %726 = insertelement <32 x float> %725, float %690, i64 25, !dbg !54 %727 = insertelement <32 x float> %726, float %689, i64 26, !dbg !54 %728 = insertelement <32 x float> %727, float %691, i64 27, !dbg !54 %729 = insertelement <32 x float> %728, float %692, i64 28, !dbg !54 %730 = insertelement <32 x float> %729, float %694, i64 29, !dbg !54 %731 = insertelement <32 x float> %730, float %693, i64 30, !dbg !54 %732 = insertelement <32 x float> %731, float %695, i64 31, !dbg !54 %733 = fptrunc <32 x float> %732 to <32 x half>, !dbg !54
br label %._crit_edge, !dbg !27